

1. Construction of binary parse tree.

- construct a binary parse tree for the formula $(\phi_1 - \phi)$ with literals as leaves and connectives as internal nodes.

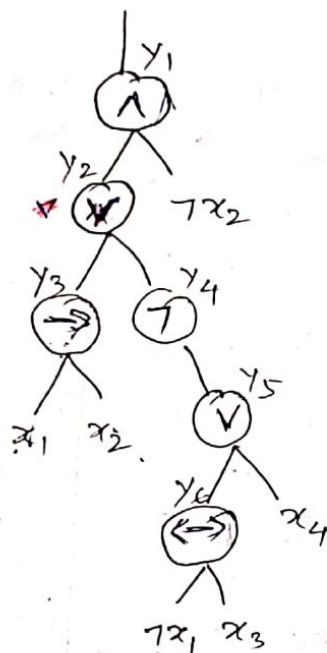
[connectives — AND, OR, NOT, $\leftrightarrow$, $\rightarrow$]

Given the formula ϕ as

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

[Note: Associativity property can be used to introduce parenthesis

$$\begin{aligned} A \wedge (B \wedge C) &= (A \wedge B) \wedge C \\ A \vee (B \vee C) &= (A \vee B) \vee C \end{aligned}$$



- contains OP of several literals
- every internal node in the resulting tree has 1 (or) 2 children
- This tree viewed as a circuit
- Introduce the variable y_i for the output of each internal node.

- Rewrite the original internal formula ϕ as the AND of the Root variable (y_1) and a conjunction of clauses ϕ_i' describing the operation of each node.

This is given by the formula ϕ'

i.e.,

$$\begin{aligned} \Phi = & Y_1 \wedge (Y_1 \leftrightarrow (Y_2 \wedge \neg X_2)) \\ & \wedge (Y_2 \leftrightarrow (Y_3 \vee Y_4)) \\ & \wedge (Y_3 \leftrightarrow (X_1 \rightarrow X_2)) \\ & \wedge (Y_4 \leftrightarrow \neg Y_5) \\ & \wedge (Y_5 \leftrightarrow (Y_6 \vee X_4)) \\ & \wedge (Y_6 \leftrightarrow (\neg X_1 \leftrightarrow X_3)) \end{aligned}$$

Y_1, Y_5	$Y_1 \leftrightarrow \neg Y_5$
1 1	$1 \leftrightarrow 0 \Rightarrow 0$
0 1	$0 \leftrightarrow 0 \Rightarrow 1 \checkmark$
0 0	$0 \leftrightarrow 1 \Rightarrow 0$
1 0	$1 \leftrightarrow 1 \Rightarrow 1 \checkmark$

The above conjunction of clauses are Φ_0' with at most 3-literals. $(Y_4 \wedge Y_5) \vee (\neg Y_4 \wedge \neg Y_5) \xrightarrow{\text{DNF}}$

2. construction of truth table. $(\neg Y_4 \vee \neg Y_5) \wedge (Y_4 \vee Y_5) \xrightarrow{\text{CNF}}$

- construct truth table for each of Φ_i' by evaluating possible assignments to its variable. $(\neg Y_4 \vee \neg Y_5 \vee Y_6)$
- In the truth table we consider only that entries that evaluates '0' and build a formula in disjunctive normal form (DNF) [i.e. OR of AND's] $(\neg Y_4 \vee \neg Y_5 \vee Y_6) \wedge (\neg Y_4 \vee Y_5 \vee Y_6)$

i.e., consider the entry in the table whose input is $Y_1 = Y_2 = X_2 = 1$, and the output 0.

Y_1, Y_2, X_2	$Y_1 \leftrightarrow (Y_2 \wedge \neg X_2)$
1, 1, 1	$1 \leftrightarrow 0 \Rightarrow 0$
1, 0, 1	$1 \leftrightarrow 0 \Rightarrow 0 \checkmark$
1, 0, 0	$1 \leftrightarrow 0 \Rightarrow 0$
0, 0, 0	$0 \leftrightarrow 0 \Rightarrow 1$
0, 1, 0	$0 \leftrightarrow 1 \Rightarrow 0 \checkmark$
0, 0, 1	$0 \leftrightarrow 0 \Rightarrow 1$
0, 1, 1	$0 \leftrightarrow 0 \Rightarrow 1$

The above entries is written as (with zero as output)

$$(Y_1 \wedge Y_2 \wedge X_2) \vee (Y_1 \wedge \neg Y_2 \wedge X_2) \vee (X_1 \wedge \neg Y_2 \wedge X_2) \vee (\neg Y_1 \wedge Y_2 \wedge \neg X_2)$$

- negating & apply Demorgan's
- we then convert this DNF formula to CNF by applying Demorgan's law (i.e., complement the literals and change OR's into AND's and AND's into OR's)
- The formula is given by Φ''

→ Replaced with:
 $\neg(P \wedge Q) \Leftrightarrow (\neg P) \vee (\neg Q)$
 $\neg(P \vee Q) \Leftrightarrow (\neg P) \wedge (\neg Q)$

$$\begin{aligned} \overline{A \cap B} &= \overline{A} \cap \overline{B} \\ \overline{A \cup B} &= \overline{A} \cup \overline{B} \end{aligned}$$

$$\Phi_1'' = (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge$$

($\neg x_1 \vee x_2 \vee x_3$) $\wedge$ ($x_1 \vee \neg x_2 \vee x_3$)
 (I/p in this will be the value of x_1, x_2, x_3 which gives, and which is equivalent to original Φ Φ output 1)

- Here the formula Φ' is reduced to Φ'' in polynomial time.

- Each Φ_i' in the previous steps reduced to CNF form by constructing respective truth table. This process takes polynomial time.

- In this step $\Phi' \rightarrow \Phi''$, which is equivalent.

3. Transforming the above CNF to 3-CNF.

- Transform the formula Φ'' , so that each clause has exactly 3 distinct literals.

- The final 3-CNF formula Φ''' is constructed from the clauses of the CNF formula Φ'' .

- Here we do the following transformation.

For each clause C_i of Φ'' , we include the following clauses in Φ''' by taking auxiliary variable p and q .

1. If C_i has 3 distinct literals then simply include C_i as a clause of Φ''' .

2. If C_i has 2 distinct literals

$C_i = (l_1 \vee l_2)$ where l_1 and l_2 are literals then include $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$ as clause of Φ'''
 $p=0$ (or) $p=1$

3. If C_i has just 1 distinct literal l_1 , then include $(l_1 \vee p \vee q) \wedge (l_1 \vee p \vee \neg q) \wedge (l_1 \vee \neg p \vee q) \wedge (l_1 \vee \neg p \vee \neg q)$ as clause of Φ'''

- 3-CNF formula ϕ''' is satisfiable iff ϕ is satisfiable. (12)

The conclusion is:

The Reduction: ϕ''' is satisfiable iff ϕ is satisfiable.

ϕ' from ϕ - this is proved by inspecting each step. preserves satisfiability

✓ ϕ'' equivalent to ϕ'

3-CNF ϕ''' is equivalent to ϕ''

To show that the reduction can be computed in polynomial time.

- Constructing ϕ' from ϕ , introduces one variable and one clause per connectivity in ϕ

- constructing ϕ'' from ϕ' introduces 8 clauses into ϕ'' for each clause from ϕ' . Since each clause of ϕ' has almost 3 variables and the truth table for each clause has almost 8 rows

- constructing ϕ''' from ϕ'' introduces at most 4 clauses in ϕ'''

$\therefore$ The above reductions in each step takes polynomial time.

The size of the resulting formula ϕ''' is poly...

Hence 3-CNF-SAT is NP complete. The length of original formula

